

实验三：语法分析

一、实验概述

本实验是编译原理实验系列的第三个实验。在实验二中，你实现了词法分析器，能够将源代码转换为 Token 序列。本实验的目标是：在此基础上实现语法分析器（Parser），将 Token 序列按照你语言的文法构建为抽象语法树（AST），并对语法错误进行检测、定位与恢复。

语法分析是编译器的核心阶段之一，它把线性的 Token 序列还原为体现程序结构的树形表示。抽象语法树是后续语义分析、中间代码生成的输入，因此它的设计直接影响整个编译器的后端。

二、实验目标

- 理解自顶向下（递归下降 / LL）与自底向上（LR）语法分析的基本原理
- 掌握上下文无关文法到语法分析器的实现方法
- 正确处理表达式的运算符优先级与结合性
- 能够构建抽象语法树（AST），并设计清晰的 AST 节点表示
- 实现语法错误的检测、定位与恢复

三、实验内容

任务一：语法分析器实现（必做）

根据你语言的语法规则（实验一中的 BNF/EBNF 文法），实现语法分析器。程序以词法分析器的输出（或直接读入源文件）为输入，构建并输出抽象语法树（JSON 格式）。

1. 命令行接口

```
./compiler parse input.src
```

成功时输出抽象语法树（JSON 格式），例如对 `if (x > 0) y = 1;` 的解析结果：

```
{
  "ast": {
    "type": "IfStmt", "line": 1, "col": 1,
    "cond": {
      "type": "BinaryExpr", "op": ">",
      "left": {"type": "Identifier", "name": "x"},
      "right": {"type": "IntLiteral", "value": 0}
    },
    "then": {
      "type": "AssignStmt",
      "target": {"type": "Identifier", "name": "y"},
      "value": {"type": "IntLiteral", "value": 1}
    }
  },
  "errors": []
}
```

存在语法错误时输出（报告期望符号与实际遇到的符号）：

```
{
  "ast": null,
  "errors": [
    {"type": "SYNTAX_ERROR", "expected": ")", "found": ";",
     "line": 1, "col": 10, "message": "expected ')' before ';'"}
  ]
}
```

2. AST 节点格式要求

每个 AST 节点必须包含 `type` 字段标识节点类型，并保留位置信息（`line`, `col`）。

常见节点类型示例：

节点类别	示例节点类型	说明
表达式	BinaryExpr, UnaryExpr, Call, Identifier, IntLiteral	运算、调用、字面量等

语句	AssignStmt, IfStmt, WhileStmt, ReturnStmt, Block	赋值、控制流、语句块等
声明	VarDecl, FuncDecl, ParamDecl	变量、函数、参数声明
程序	Program	根节点，包含顶层声明/语句列表

AST 的具体结构由你根据自己的语言设计，但必须文档化（在实验报告中给出节点定义），并保证同一类结构生成一致的节点形态，便于后续实验复用。

3. 必做功能

(1) 完整覆盖语言文法

- 能够解析你语言定义的全部语法结构：表达式、语句、声明、函数/块结构等
- 正确处理嵌套结构（嵌套表达式、嵌套语句块、嵌套控制流）

(2) 运算符优先级与结合性

- 正确处理算术、关系、逻辑运算符的优先级，例如 $a + b * c$ 应解析为 $a + (b * c)$
- 正确处理结合性，例如减法左结合 $a - b - c$ 解析为 $(a - b) - c$
- 正确处理括号改变优先级

(3) 语法错误检测与定位

当 Token 序列不符合文法时，必须报告语法错误，给出期望的符号、实际遇到的符号及其位置。

(4) 错误恢复

采用错误恢复策略（如 panic-mode：跳过 Token 直到遇到同步符号，如分号、右括号），使分析器在遇到一个错误后能够继续分析，一次尽可能多地报告语法错误，而不是遇到第一个错误就停止。

(5) 分析方法

分析方法由你自选（递归下降 / LL(1) 预测分析 / LR 等），鼓励手写实现以加深理解。若使用分析器生成工具，需在报告中说明，并仍需完整解释其工作原理。

任务二：测试用例集（必做）

编写测试用例验证语法分析器的正确性，并在报告与演示中展示。要求 ≥ 10 组，覆盖以下场景：

编号	测试场景	最少用例数
T1	表达式优先级与结合性	2
T2	各类语句（赋值、控制流、声明）	2
T3	嵌套结构（嵌套块、嵌套表达式）	2
T4	函数/过程定义与调用（如有）	1
T5	语法错误检测与错误恢复（缺括号、缺分号、非法结构等）	3

任务三：必考场景验证（必做）

使用必考场景程序验证语法分析器（场景定义见《必考场景集（扩充版）》）：

- S1 - S5 的程序必须成功解析，生成正确的 AST，无语法错误
 - S7（词法合法、含语法错误的场景）的语法错误必须被正确检测、定位并报告
- 在演示与报告中展示每个场景程序的 AST 输出（或语法树图示）。

任务四：语法树可视化（选做，加分项）

实现将 AST 导出为可渲染格式（如 Graphviz DOT），生成语法树的图形表示；或针对你语言中存在二义性的文法，说明你是如何消解二义性的（如优先级声明、文法改写）。

```
./compiler parse input.src --dump-dot
```

四、交付物清单

序号	交付物	文件/格式	说明
1	语法分析器源代码	源代码文件	建议与实验二共用同一代码库
2	构建/运行说明	README.md	如何编译和运行
3	AST 节点定义文档	ast-design.md 或写入报告	节点类型与结构说明
4	测试用例集	tests/test_XX.src + .expected.json	≥ 10 组
5	场景 parse 结果	scenarios/S1.ast.json ~ S5.ast.json + S7.ast.json	正向 5 个 + 语法错误场景 S7
6	AI 协作对话记录	conversation/ 目录 (Markdown 或导出文件)	完整对话 + 每轮工作纪要
7	实验报告	report.pdf / report.docx	含完整功能演示流程 (图文)

五、验收方式

本实验采用过程化验收，重点考察真实的开发过程以及功能实现的一致性，而非仅看最终代码。验收由助教完成，包含以下四个环节。四个环节相互印证：对话记录、实验报告、源代码三者必须能够对应起来。

环节一：实验演示检查

- 学生现场演示（或提交完整录屏），展示编译器本阶段功能的完整运行流程，覆盖本实验全部必做功能。

- 演示需同时体现：正常输入的正确处理，以及错误输入的检测与报告。
- 助教可在演示过程中临时更换输入用例、要求你解释某段代码的作用或某个设计决策的理由。

环节二：AI 协作过程记录检查

- 提交开发全过程中与智能体（Agent IDE / AI 助手，如 Claude Code、Cursor、CodeBuddy 等）的完整对话记录。
- 记录需包含每一轮交互的三部分：（1）你向智能体提出的问题或指令；（2）智能体对该轮对话的完整响应；（3）该轮的工作纪要——你做了什么决策、为什么这样做、如何验证结果。
- 工作纪要不要求包含代码输出本身，重点是设计思路、决策理由与验证过程。
- 课程鼓励使用 AI 工具，但你必须能够独立解释每一行代码与每一个设计选择。完整、连贯的对话记录是你真正参与开发的证据。

环节三：实验报告检查

- 实验报告必须包含完整的功能演示流程（图文并茂）：对每一项功能，给出运行截图加文字说明，完整展示从输入到输出的过程。
- 报告还需涵盖：设计决策、关键算法实现思路、遇到的问题及其解决方法。

环节四：代码审核

- 助教审核源代码，要求与环节二的对话记录、环节三的报告演示流程保持一致：
 - 代码中实现的功能，应能在报告的演示流程中找到对应的展示；
 - 代码的实现方式与思路，应能由对话记录与工作纪要解释其来源。
- 若出现明显不一致（例如代码中存在报告未演示、对话记录也无法解释的功能），将影响本实验成绩。

六、实验报告要求

实验报告是验收的核心材料之一，必须包含以下内容：

（1）完整功能演示流程（图文并茂）

- 逐项展示本实验的每个功能：给出输入示例、运行命令、运行截图与输出结果，并配文字说明。
- 演示流程需与源代码、AI 对话记录一致（详见第五部分验收方式）。

（2）设计与实现说明

- 你的 AST 是如何设计的？给出主要节点类型的定义。
- 运算符优先级与结合性是如何实现的？举例说明。
- 你采用了哪种语法分析方法？为什么选择它？
- 语法错误的检测与恢复策略是什么？举例说明一次输入触发多个错误的情况。
- 语法分析器如何与词法分析器（实验二）、后续语义分析（实验四）对接。

（3）AI 协作过程小结

- 简述你如何与智能体协作完成本实验：哪些部分由你主导设计、哪些借助 AI 实现、你如何验证 AI 产出的正确性。

七、参考资料

- Aho, Lam, Sethi, Ullman. Compilers: Principles, Techniques, and Tools (Dragon Book), Chapter 4
- Appel. Modern Compiler Implementation, Chapter 3
- Grune, Jacobs. Parsing Techniques: A Practical Guide, Chapter 6 - 9
- Yacc/Bison、ANTLR 等语法分析器生成工具的文档（仅供参考）